CO2 AT2 – Function Calling

Name: B. Paneesha

Reg. No.: 192472206

Course code: CSA6502

Course Name: Generative AI

Question 24: API-Based Structured Information Extraction Using Function Calling

Problem Statement

A company wants to use an LLM API to automatically extract customer information from emails. The extracted information should be returned in a structured JSON format so it can be directly stored in databases or CRM systems.

1. Aim

To design a **function-calling workflow** that extracts customer information from emails using an LLM API and returns a predictable JSON object through structured output.

Objectives

- Extract customer details automatically.
- Reduce manual effort.
- Generate structured JSON output.
- Enable database integration.
- Improve processing speed and accuracy.

2. Prompt

You are an intelligent information extraction assistant.

Read the customer email carefully and extract the following details:

• Customer Name

• Customer ID

• Email Address

• Phone Number

• Address

• Request Type

If any information is unavailable, return null.

Return the result only by calling the function

extract_customer_information().

Do not generate normal text.

3. Function Calling Schema

Include:

- Function Name
- Purpose
- Parameters
- Required Fields
- Data Types
- JSON Function Schema

Function Name

extract_customer_information()

Purpose

Extracts customer details from an email and returns structured information.

Parameters

Parameter	Type	Required	Description
email_text	string	Yes	Customer email content
customer_name	string	Yes	Name of customer
customer_id	string	Yes	Customer identification number
email	string	Yes	Email address

Parameter	Type	Required	Description
phone	string	No	Phone number
address	string	No	Postal address
request_type	string	Yes	Type of customer request

JSON Function Schema

```
{
 "name": "extract_customer_information",
 "description": "Extract customer information from email",
 "parameters": {
  "type": "object",
  "properties": {
   "email_text": {
    "type": "string",
    "description": "Customer email content"
   },
   "customer_name": {
    "type": "string",
    "description": "Customer full name"
   },
   "customer_id": {
    "type": "string",
    "description": "Customer ID"
   },
   "email": {
    "type": "string",
    "description": "Customer email address"
   },
```

```
      "phone": {
        "type": "string",
        "description": "Phone number"
      },
      "address": {
        "type": "string",
        "description": "Customer address"
      },
      "request_type": {
        "type": "string",
        "description": "Customer request type"
      }
    },
    "required": [
      "email_text",
      "customer_name",
      "customer_id",
      "email",
      "request_type"
    ]
  }
}
```

4. Structured Output Schema

Example Output

```
{
  "customer_name": "John Smith",
  "customer_id": "CUST-45231",
  "email": "john.smith@email.com",
```

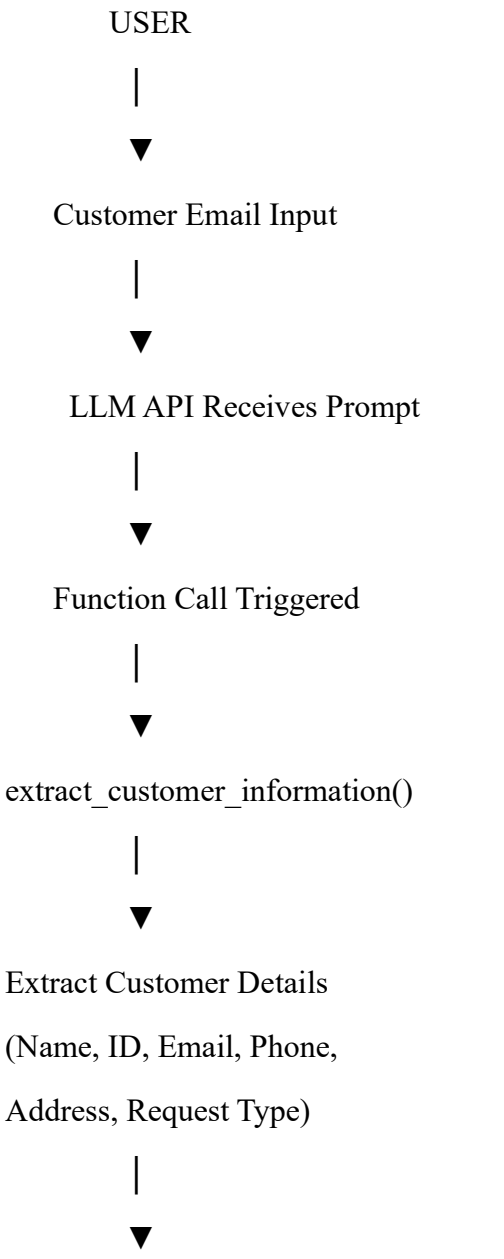
```
  "phone": "+1-212-555-7890",
  "address": "45 Park Avenue, New York, NY 10016",
  "request_type": "Address Update"
}
```


JSON Schema


```
{
  "type": "object",
  "properties": {
    "customer_name": {
      "type": "string"
    },
    "customer_id": {
      "type": "string"
    },
    "email": {
      "type": "string"
    },
    "phone": {
      "type": "string"
    },
    "address": {
      "type": "string"
    },
    "request_type": {
      "type": "string"
    }
  },
  "required": [
    "customer_name",
```

```
    "customer_id",
    "email",
    "request_type"
  ]
}
```

5. Function Calling Workflow



6. Python Implementation

```
File  Edit  Format  Run  Options  Window  Help
import re
import json

# ------------------------------
# Sample Customer Email
# ------------------------------
email_text = """
Subject: Address Update

Hello,

My name is John Smith.
Customer ID: CUST-45231

Please update my address to:
45 Park Avenue,
New York, NY 10016.

Phone: +1-212-555-7890
Email: john.smith@email.com

Thank you.
"""

# ------------------------------
# Function Calling Simulation
# ------------------------------
def extract_customer_information(email):

    # Customer Name
    name = None
    match = re.search(r"My name is ([A-Za-z ]+)", email)
    if match:
        name = match.group(1).strip()

    # Customer ID
    customer_id = None
    match = re.search(r"Customer ID:\s*([A-Za-z0-9\-]+)", email)
    if match:
        customer_id = match.group(1)

    # Email
    email_id = None
    match = re.search(r'[\w\.-]+@[\w\.-]+\.\w+', email)
    if match:
        email_id = match.group()
```

Output :

```
IDLE Shell 3.14.3
File  Edit  Shell  Debug  Options  Window  Help
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/panee/OneDrive/Documents/gvfg.py ==========
==========
============================================================
API-Based Structured Information Extraction
============================================================

Structured JSON Output:

{
    "customer_name": "John Smith",
    "customer_id": "CUST-45231",
    "email": "john.smith@email.com",
    "phone": "+1-212-555-7890",
    "address": "45 Park Avenue, New York, NY 10016.",
    "request_type": "Address Update"
}

Program Executed Successfully.
>>>
========== RESTART: C:/Users/panee/OneDrive/Documents/gvfg.py ==========
============================================================
API-Based Structured Information Extraction
============================================================

Structured JSON Output:

{
    "customer_name": "John Smith",
    "customer_id": "CUST-45231",
    "email": "john.smith@email.com",
    "phone": "+1-212-555-7890",
    "address": "45 Park Avenue, New York, NY 10016.",
    "request_type": "Address Update"
}

Program Executed Successfully.
>>>
Ln: 39  Col: 0
```

7. Test Cases

Test Case 1

Email

My name is Alice Brown.

Customer ID: CUST-1001

Email: alice@gmail.com

Phone: 9876543210

Please update my address.

Output


```
{
  "customer_name":"Alice Brown",
  "customer_id":"CUST-1001",
  "email":"alice@gmail.com",
  "phone":"9876543210",
  "address":null,
  "request_type":"Address Update"
}
```


Test Case 2

Email

Customer ID: CUST-5678

My internet is not working.

Output


```
{
  "customer_name":null,
  "customer_id":"CUST-5678",
```

"email":null,
"phone":null,
"address":null,
"request_type":"Technical Support"
}

Test Case 3

Email

My name is Sarah.

Customer ID: CUST-9988

I want a refund.

Email: sarah@email.com

Output

{
"customer_name":"Sarah",
"customer_id":"CUST-9988",
"email":"sarah@email.com",
"phone":null,
"address":null,
"request_type":"Refund"
}

Test Case 4

Email

John Smith

Customer ID: CUST-9000

Phone: 9876543210

My package arrived damaged.

Output


```
{
  "customer_name":"John Smith",
  "customer_id":"CUST-9000",
  "email":null,
  "phone":"9876543210",
  "address":null,
  "request_type":"Complaint"
}
```


Test Case 5

Email

Please close my account.

Customer ID: CUST-1111

Email: user@test.com

Output


```
{
  "customer_name":null,
  "customer_id":"CUST-1111",
  "email":"user@test.com",
  "phone":null,
  "address":null,
  "request_type":"Account Closure"
}
```


8. Edge Cases

Missing Email

Input

email_text=""

Output

{

"error":"Email content is missing."

}

Invalid Data Type

Input

email_text=12345

Output

{

"error":"Email content must be a string."

}

No Customer Information Found

Input

Hello

Thank you.

Output

{

"error":"No customer information found."

}

9. Design Justification

- **Function Calling:** Separates the extraction logic from text generation, making the workflow modular and reusable.
- **Structured Output:** Ensures every API response follows the same JSON format, enabling direct integration with CRM systems and databases.
- **Validation:** Required parameters and data type checks prevent invalid or incomplete function calls.

- **Consistency:** A fixed schema guarantees predictable outputs across different customer emails.
- **Edge-Case Handling:** Handles missing fields, invalid input types, and incomplete emails gracefully without producing malformed JSON.

10. Result

The designed function-calling workflow successfully extracts customer information from unstructured emails and converts it into a **predictable JSON object**. Compared to free-text responses, structured output improves **accuracy, consistency, reliability, and ease of integration** with downstream applications such as CRM systems, customer support platforms, and databases. This makes the solution suitable for automating customer information extraction at scale.